Mutable Variables

CS 350

Dr. Joseph Eremondi

May 7, 2025

Outline

Curly-Mutvar

Curly-Mutvar

Overview

- Learning goals

Overview

- Learning goals
  - To interpret a language where variables can change values (mutate)

Overview

- Learning goals
  - To interpret a language where variables can change values (mutate)
  - To understand the design choices around functions in such a language

Overview

- Learning goals
  - To interpret a language where variables can change values (mutate)
  - To understand the design choices around functions in such a language
- Key concepts

Overview

- Learning goals
  - To interpret a language where variables can change values (mutate)
  - To understand the design choices around functions in such a language
- Key concepts
  - Pass-by-reference vs. Pass-by-value

Identifiers vs Variables

- Until now, a variable denoted a *value*

Identifiers vs Variables

- Until now, a variable denoted a *value*
  - In a given environment

Identifiers vs Variables

- Until now, a variable denoted a *value*
  - In a given environment
- They didn’t really ever vary

Identifiers vs Variables

- Until now, a variable denoted a *value*
  - In a given environment
- They didn’t really ever vary
  - Except between different function calls

Identifiers vs Variables

- Until now, a variable denoted a *value*
  - In a given environment
- They didn't really ever vary
  - Except between different function calls
- To allow variables values to change, we can make one simple change:

Identifiers vs Variables

- Until now, a variable denoted a *value*
  - In a given environment
- They didn't really ever vary
  - Except between different function calls
- To allow variables values to change, we can make one simple change:
  - **Keep locations instead of values in the environment**

Identifiers vs Variables

- Until now, a variable denoted a *value*
  - In a given environment
- They didn't really ever vary
  - Except between different function calls
- To allow variables values to change, we can make one simple change:
  - **Keep locations instead of values in the environment**
  - Then each variable refers to a single store location, whose value can change

Curly-Mutvar

- Extend Curly-Box with `{set-var! VARIABLE <expr>}`

Curly-Mutvar

- Extend Curly-Box with `{set-var! VARIABLE <expr>}`
- E.g.

Curly-Mutvar

- Extend Curly-Box with `{set-var! VARIABLE <expr>}`
- E.g.

Curly-Mutvar

- Extend Curly-Box with `{set-var! VARIABLE <expr>}`
- E.g.

```
{let1 {x 3}
      {seq {set-var! x {* x 2}}
           x}}
```

- x starts out as 3

Curly-Mutvar

- Extend Curly-Box with `{set-var! VARIABLE <expr>}`
- E.g.

```
{let1 {x 3}
      {seq {set-var! x {* x 2}}
           x}}
```

- `x` starts out as 3
- `set-var!` changes reads the old value `3`, then sets the value of `x` to double this

Curly-Mutvar

- Extend Curly-Box with `{set-var! VARIABLE <expr>}`
- E.g.

```
{let1 {x 3}
      {seq {set-var! x {* x 2}}
           x}}
```

- `x` starts out as 3
- `set-var!` changes reads the old value `3`, then sets the value of `x` to double this
- End result is 6, the new value for `x`

Bindings with mutable variables

- Each binding associates a symbol with a *location*

Bindings with mutable variables

- Each binding associates a symbol with a *location*

Bindings with mutable variables

- Each binding associates a symbol with a *location*

```
(define-type Binding (Symbol * Location))
```

- All other environment operations are the same

Bindings with mutable variables

- Each binding associates a symbol with a *location*

```
(define-type Binding (Symbol * Location))
```

- All other environment operations are the same
- Lookup for environments now has type:

Bindings with mutable variables

- Each binding associates a symbol with a *location*

```
(define-type Binding (Symbol * Location))
```

- All other environment operations are the same
- Lookup for environments now has type:

Bindings with mutable variables

- Each binding associates a symbol with a *location*

```
(define-type Binding (Symbol * Location))
```

- All other environment operations are the same
- Lookup for environments now has type:

```
(define (env-lookup [n : Symbol] [env : Env]) : Location ....)
```

Variables as a Two-Step Process

- Getting or setting a variable now involves two lookups:

Variables as a Two-Step Process

- Getting or setting a variable now involves two lookups:
  - Convert the variable name (symbol) into a `Location`

Variables as a Two-Step Process

- Getting or setting a variable now involves two lookups:
  - Convert the variable name (symbol) into a `Location`
    - By looking up in the environment

Variables as a Two-Step Process

- Getting or setting a variable now involves two lookups:
  - Convert the variable name (symbol) into a `Location`
    - By looking up in the environment
  - Convert the resulting location into a `Value`

Variables as a Two-Step Process

- Getting or setting a variable now involves two lookups:
  - Convert the variable name (symbol) into a `Location`
    - By looking up in the environment
  - Convert the resulting location into a `Value`
    - By looking up in the store

Variables as a Two-Step Process

- Getting or setting a variable now involves two lookups:
  - Convert the variable name (symbol) into a `Location`
    - By looking up in the environment
  - Convert the resulting location into a `Value`
    - By looking up in the store
- Static vs. Dynamic

Variables as a Two-Step Process

- Getting or setting a variable now involves two lookups:
  - Convert the variable name (symbol) into a `Location`
    - By looking up in the environment
  - Convert the resulting location into a `Value`
    - By looking up in the store
- Static vs. Dynamic
  - What variables are in scope is a *static* property

Variables as a Two-Step Process

- Getting or setting a variable now involves two lookups:
  - Convert the variable name (symbol) into a `Location`
    - By looking up in the environment
  - Convert the resulting location into a `Value`
    - By looking up in the store
- Static vs. Dynamic
  - What variables are in scope is a *static* property
    - Will always be the same for each call to a given function

Variables as a Two-Step Process

- Getting or setting a variable now involves two lookups:
  - Convert the variable name (symbol) into a `Location`
    - By looking up in the environment
  - Convert the resulting location into a `Value`
    - By looking up in the store
- Static vs. Dynamic
  - What variables are in scope is a *static* property
    - Will always be the same for each call to a given function
  - What values are in memory is a *dynamic* property

Variables as a Two-Step Process

- Getting or setting a variable now involves two lookups:
  - Convert the variable name (symbol) into a `Location`
    - By looking up in the environment
  - Convert the resulting location into a `Value`
    - By looking up in the store
- Static vs. Dynamic
  - What variables are in scope is a *static* property
    - Will always be the same for each call to a given function
  - What values are in memory is a *dynamic* property
    - Depends on every bit of code that has run before

Interpreting Variables

Interpreting Variables

```
(define (interp [env : Env]
                [e : Exp]
                [sto : Store]) : Result
  (type-case Exp e
    [(varE x)
     (v*s (lookup (lookup x env) sto) sto)]
....))
```

- Previously, we just looked up a value with `lookup`

Interpreting Variables

```
(define (interp [env : Env]
                [e : Exp]
                [sto : Store]) : Result
  (type-case Exp e
    [(varE x)
     (v*s (lookup (lookup x env) sto) sto)]
....))
```

- Previously, we just looked up a value with `lookup`
- Now we lookup a *location*

Interpreting Variables

```
(define (interp [env : Env]
                [e : Exp]
                [sto : Store]) : Result
  (type-case Exp e
    [(varE x)
     (v*s (lookup (lookup x env) sto) sto)]
....))
```

- Previously, we just looked up a value with `lookup`
- Now we lookup a *location*
  - Have to use `lookup` again to get its value from the store

Interpreting Variables

```
(define (interp [env : Env]
                [e : Exp]
                [sto : Store]) : Result
  (type-case Exp e
    [(varE x)
     (v*s (lookup (lookup x env) sto) sto)]
....))
```

- Previously, we just looked up a value with `lookup`
- Now we lookup a *location*
  - Have to use `lookup` again to get its value from the store
    - Recall, `lookup` is polymorphic

Interpreting Variables

```
(define (interp [env : Env]
                [e : Exp]
                [sto : Store]) : Result
  (type-case Exp e
    [(varE x)
     (v*s (lookup (lookup x env) sto) sto)]
....))
```

- Previously, we just looked up a value with `lookup`
- Now we lookup a *location*
  - Have to use `lookup` again to get its value from the store
    - Recall, `lookup` is polymorphic
    - `lookup : ('key (AssocList 'key 'value) -> 'value)`

Interpreting Variables

```
(define (interp [env : Env]
                [e : Exp]
                [sto : Store]) : Result
  (type-case Exp e
    [(varE x)
     (v*s (lookup (lookup x env) sto) sto)]
....))
```

- Previously, we just looked up a value with `lookup`
- Now we lookup a *location*
  - Have to use `lookup` again to get its value from the store
    - Recall, `lookup` is polymorphic
    - `lookup : ('key (AssocList 'key 'value) -> 'value)`
    - So `lookup` on env gives `Location`, lookup on store gives `Value`

Interpreting Variables

```
(define (interp [env : Env]
                [e : Exp]
                [sto : Store]) : Result
  (type-case Exp e
    [(varE x)
     (v*s (lookup (lookup x env) sto) sto)]
....))
```

- Previously, we just looked up a value with `lookup`
- Now we lookup a *location*
  - Have to use `lookup` again to get its value from the store
    - Recall, `lookup` is polymorphic
    - `lookup : ('key (AssocList 'key 'value) -> 'value)`
    - So `lookup` on env gives `Location`, lookup on store gives `Value`
- Produce that value, along with the unchanged store

Syntax for Mutable Variables

- `{set-var!  SYMBOL <expr>}`

Syntax for Mutable Variables

- `{set-var!  SYMBOL <expr>}`
  - Sets the value of an in-scope variable

Syntax for Mutable Variables

- `{set-var! SYMBOL <expr>}`
  - Sets the value of an in-scope variable
  - Just like `set-box!` except we get the location from the environment, instead of by evaluating a box

Syntax for Mutable Variables

- `{set-var! SYMBOL <expr>}`
  - Sets the value of an in-scope variable
  - Just like `set-box!` except we get the location from the environment, instead of by evaluating a box
- Can only set the value for a specific variable

Syntax for Mutable Variables

- `{set-var! SYMBOL <expr>}`
  - Sets the value of an in-scope variable
  - Just like `set-box!` except we get the location from the environment, instead of by evaluating a box
- Can only set the value for a specific variable
  - Unlike boxes, which can be created/computed at run-time

Syntax for Mutable Variables

- `{set-var! SYMBOL <expr>}`
  - Sets the value of an in-scope variable
  - Just like `set-box!` except we get the location from the environment, instead of by evaluating a box
- Can only set the value for a specific variable
  - Unlike boxes, which can be created/computed at run-time
- New AST constructor:

Syntax for Mutable Variables

- `{set-var! SYMBOL <expr>}`
  - Sets the value of an in-scope variable
  - Just like `set-box!` except we get the location from the environment, instead of by evaluating a box
- Can only set the value for a specific variable
  - Unlike boxes, which can be created/computed at run-time
- New AST constructor:

Syntax for Mutable Variables

- `{set-var! SYMBOL <expr>}`
  - Sets the value of an in-scope variable
  - Just like `set-box!` except we get the location from the environment, instead of by evaluating a box
- Can only set the value for a specific variable
  - Unlike boxes, which can be created/computed at run-time
- New AST constructor:

```
(define-type Exp
  ...
  (set-var!E [var : Symbol]
             [e : Exp]))
```

Interpreting Mutation

Interpreting Mutation

```
(define (interp [env : Env]
                [e : Exp]
                [sto : Store]) : Result
(type-case Exp e
  [(set-varE! x e)
    (with ([e-val e-sto] (interp env e sto))
      (v*s e-val
        ;; Get the location from the environment
        (override-store (lookup x env)
                        e-value
                        e-sto)))]
....))
```

- Interpret the new value for the variable

Interpreting Mutation

```
(define (interp [env : Env]
                [e : Exp]
                [sto : Store]) : Result
(type-case Exp e
  [(set-varE! x e)
    (with ([e-val e-sto] (interp env e sto))
      (v*s e-val
        ;; Get the location from the environment
        (override-store (lookup x env)
                        e-value
                        e-sto)))]
....))
```

- Interpret the new value for the variable
  - This gives a value and a new store

Interpreting Mutation

```
(define (interp [env : Env]
                [e : Exp]
                [sto : Store]) : Result
(type-case Exp e
  [(set-varE! x e)
    (with ([e-val e-sto] (interp env e sto))
      (v*s e-val
        ;; Get the location from the environment
        (override-store (lookup x env)
                        e-value
                        e-sto)))]
....))
```

- Interpret the new value for the variable
  - This gives a value and a new store
- Lookup up the location for the variable

Interpreting Mutation

```
(define (interp [env : Env]
                [e : Exp]
                [sto : Store]) : Result
(type-case Exp e
  [(set-varE! x e)
    (with ([e-val e-sto] (interp env e sto))
      (v*s e-val
        ;; Get the location from the environment
        (override-store (lookup x env)
                        e-value
                        e-sto)))]
....))
```

- Interpret the new value for the variable
  - This gives a value and a new store
- Lookup up the location for the variable
- Return a store with the new value at that location

Function Calls

- To call a function, we evaluate the body in the environment extended with the argument value

Function Calls

- To call a function, we evaluate the body in the environment extended with the argument value
  - Environment now takes *locations*

Function Calls

- To call a function, we evaluate the body in the environment extended with the argument value
  - Environment now takes *locations*
  - Need a location to associate with the new variable

Function Calls

- To call a function, we evaluate the body in the environment extended with the argument value
  - Environment now takes *locations*
  - Need a location to associate with the new variable
  - Need to make sure the argument value ends up at that location

Function Calls

- To call a function, we evaluate the body in the environment extended with the argument value
  - Environment now takes *locations*
  - Need a location to associate with the new variable
  - Need to make sure the argument value ends up at that location
- If the argument is e.g. a number, then we have to make a new location for it

Function Calls

- To call a function, we evaluate the body in the environment extended with the argument value
  - Environment now takes *locations*
  - Need a location to associate with the new variable
  - Need to make sure the argument value ends up at that location
- If the argument is e.g. a number, then we have to make a new location for it
- *What if the argument is already a variable?*

Function Calls

- To call a function, we evaluate the body in the environment extended with the argument value
  - Environment now takes *locations*
  - Need a location to associate with the new variable
  - Need to make sure the argument value ends up at that location
- If the argument is e.g. a number, then we have to make a new location for it
- *What if the argument is already a variable?*
  - Have a design decision

Pass-by-value

- Default in C/C++

Pass-by-value

- Default in C/C++
- If we implement function calls using pass-by-value, then:

Pass-by-value

- Default in C/C++
- If we implement function calls using pass-by-value, then:
  - **Each function call generates a new location where its argument values are stored**

Pass-by-value

- Default in C/C++
- If we implement function calls using pass-by-value, then:
  - **Each function call generates a new location where its argument values are stored**
  - If the arguments are variables, their values are looked up and copied to the new location

Pass-by-value

- Default in C/C++
- If we implement function calls using pass-by-value, then:
  - **Each function call generates a new location where its argument values are stored**
  - If the arguments are variables, their values are looked up and copied to the new location
    - e.g. the normal way to interpret a variable

Pass-by-value

- Default in C/C++
- If we implement function calls using pass-by-value, then:
  - **Each function call generates a new location where its argument values are stored**
  - If the arguments are variables, their values are looked up and copied to the new location
    - e.g. the normal way to interpret a variable
- Each function call has:

Pass-by-value

- Default in C/C++
- If we implement function calls using pass-by-value, then:
  - **Each function call generates a new location where its argument values are stored**
  - If the arguments are variables, their values are looked up and copied to the new location
    - e.g. the normal way to interpret a variable
- Each function call has:
  - A new symbol in the environment (the parameter)

Pass-by-value

- Default in C/C++
- If we implement function calls using pass-by-value, then:
  - **Each function call generates a new location where its argument values are stored**
  - If the arguments are variables, their values are looked up and copied to the new location
    - e.g. the normal way to interpret a variable
- Each function call has:
  - A new symbol in the environment (the parameter)
  - Pointing to a new location

Pass-by-value interp

Pass-by-value interp

```
[(appE funExpr argExpr)
     (with ([fun-v fun-sto] (interp env funExpr sto))
           ([arg-v arg-sto] (interp env argExpr fun-sto))
           (type-case Value fun-v
             [(closureV argVar funBody funEnv)
                (let* ([argLoc (new-loc arg-sto)]
                       [body-sto (override-store argLoc arg-v arg-sto)])
                (interp (extend argVar argLoc funEnv)
                        funBody
                        body-sto))]
             [else error 'interp "Tried to call non-function")]))]
```

- Evaluate function and arg

Pass-by-value interp

```
[(appE funExpr argExpr)
     (with ([fun-v fun-sto] (interp env funExpr sto))
           ([arg-v arg-sto] (interp env argExpr fun-sto))
           (type-case Value fun-v
             [(closureV argVar funBody funEnv)
                (let* ([argLoc (new-loc arg-sto)]
                       [body-sto (override-store argLoc arg-v arg-sto)])
                (interp (extend argVar argLoc funEnv)
                        funBody
                        body-sto))]
             [else error 'interp "Tried to call non-function")]))]
```

- Evaluate function and arg
  - Gives value and stores, threaded linearly

Pass-by-value interp

```
[(appE funExpr argExpr)
     (with ([fun-v fun-sto] (interp env funExpr sto))
           ([arg-v arg-sto] (interp env argExpr fun-sto))
           (type-case Value fun-v
             [(closureV argVar funBody funEnv)
                (let* ([argLoc (new-loc arg-sto)]
                       [body-sto (override-store argLoc arg-v arg-sto)])
                (interp (extend argVar argLoc funEnv)
                        funBody
                        body-sto))]
             [else error 'interp "Tried to call non-function")]))]
```

- Evaluate function and arg
  - Gives value and stores, threaded linearly
- New location `argLoc` generated for function parameter

Pass-by-value interp

```
[(appE funExpr argExpr)
     (with ([fun-v fun-sto] (interp env funExpr sto))
           ([arg-v arg-sto] (interp env argExpr fun-sto))
           (type-case Value fun-v
             [(closureV argVar funBody funEnv)
                (let* ([argLoc (new-loc arg-sto)]
                       [body-sto (override-store argLoc arg-v arg-sto)])
                (interp (extend argVar argLoc funEnv)
                        funBody
                        body-sto))]
             [else error 'interp "Tried to call non-function")]))]
```

- Evaluate function and arg
  - Gives value and stores, threaded linearly
- New location `argLoc` generated for function parameter
  - New store `body-sto` with Parameter value added to this location

Pass-by-value interp

```
[(appE funExpr argExpr)
     (with ([fun-v fun-sto] (interp env funExpr sto))
           ([arg-v arg-sto] (interp env argExpr fun-sto))
           (type-case Value fun-v
             [(closureV argVar funBody funEnv)
                (let* ([argLoc (new-loc arg-sto)]
                       [body-sto (override-store argLoc arg-v arg-sto)])
                (interp (extend argVar argLoc funEnv)
                        funBody
                        body-sto))]
             [else error 'interp "Tried to call non-function")]))]
```

- Evaluate function and arg
  - Gives value and stores, threaded linearly
- New location `argLoc` generated for function parameter
  - New store `body-sto` with Parameter value added to this location
- Evaluate function body

Pass-by-value interp

```
[(appE funExpr argExpr)
     (with ([fun-v fun-sto] (interp env funExpr sto))
           ([arg-v arg-sto] (interp env argExpr fun-sto))
           (type-case Value fun-v
             [(closureV argVar funBody funEnv)
                (let* ([argLoc (new-loc arg-sto)]
                       [body-sto (override-store argLoc arg-v arg-sto)])
                (interp (extend argVar argLoc funEnv)
                        funBody
                        body-sto))]
             [else error 'interp "Tried to call non-function")]))]
```

- Evaluate function and arg
  - Gives value and stores, threaded linearly
- New location `argLoc` generated for function parameter
  - New store `body-sto` with Parameter value added to this location
- Evaluate function body
  - In env from the closure

Pass-by-value interp

```
[(appE funExpr argExpr)
     (with ([fun-v fun-sto] (interp env funExpr sto))
           ([arg-v arg-sto] (interp env argExpr fun-sto))
           (type-case Value fun-v
             [(closureV argVar funBody funEnv)
                (let* ([argLoc (new-loc arg-sto)]
                       [body-sto (override-store argLoc arg-v arg-sto)])
                (interp (extend argVar argLoc funEnv)
                        funBody
                        body-sto))]
             [else error 'interp "Tried to call non-function")]))]
```

- Evaluate function and arg
  - Gives value and stores, threaded linearly
- New location `argLoc` generated for function parameter
  - New store `body-sto` with Parameter value added to this location
- Evaluate function body
  - In env from the closure
  - Extended with new location for `var`

Pass-by-value interp

```
[(appE funExpr argExpr)
     (with ([fun-v fun-sto] (interp env funExpr sto))
           ([arg-v arg-sto] (interp env argExpr fun-sto))
           (type-case Value fun-v
             [(closureV argVar funBody funEnv)
                (let* ([argLoc (new-loc arg-sto)]
                       [body-sto (override-store argLoc arg-v arg-sto)])
                (interp (extend argVar argLoc funEnv)
                        funBody
                        body-sto))]
             [else error 'interp "Tried to call non-function")]))]
```

- Evaluate function and arg
  - Gives value and stores, threaded linearly
- New location `argLoc` generated for function parameter
  - New store `body-sto` with Parameter value added to this location
- Evaluate function body
  - In env from the closure
  - Extended with new location for `var`
  - With the store containing `arg-v` at location `argLoc`

Pass-by-reference

- Pass-by-reference means:

Pass-by-reference

- Pass-by-reference means:
  - when function argument is a variable

Pass-by-reference

- Pass-by-reference means:
  - when function argument is a variable
  - function's body is evaluated in an environment *where the argument variable is bound to the input variable's location*

Pass-by-reference

- Pass-by-reference means:
  - when function argument is a variable
  - function's body is evaluated in an environment *where the argument variable is bound to the input variable's location*
- Changes to one will be seen in the other

Interpreting

Interpreting

```
;; Everything the same as pass-by-value
;; except we check if the argument is a variable
;; and use its location instead of the new one if it us
(type-case Exp argExpr
  [(varE x)
     (interp (extend argVar (lookup x env) funEnv)
                  funBody
                  arg-sto)]
  ;; Otherwise do the same as call-by-value
  [else ....])
```

- In the variable case, we don't allocate a new location

Interpreting

```
;; Everything the same as pass-by-value
;; except we check if the argument is a variable
;; and use its location instead of the new one if it us
(type-case Exp argExpr
  [(varE x)
     (interp (extend argVar (lookup x env) funEnv)
                  funBody
                  arg-sto)]
  ;; Otherwise do the same as call-by-value
  [else ....])
```

- In the variable case, we don't allocate a new location
  - Evaluate function body in closure environment

Interpreting

```
;; Everything the same as pass-by-value
;; except we check if the argument is a variable
;; and use its location instead of the new one if it us
(type-case Exp argExpr
  [(varE x)
     (interp (extend argVar (lookup x env) funEnv)
                  funBody
                  arg-sto)]
  ;; Otherwise do the same as call-by-value
  [else ....])
```

- In the variable case, we don't allocate a new location
  - Evaluate function body in closure environment
  - Extended to have parameter symbol mapped to **the variable's exiting location**

The Difference

- Any changes made to the parameter variable are lost in pass-by-value, but kept in pass-by-reference

The Difference

- Any changes made to the parameter variable are lost in pass-by-value, but kept in pass-by-reference
- Pass by value means that a function can only mutate locations that it is *explicitly given*

The Difference

- Any changes made to the parameter variable are lost in pass-by-value, but kept in pass-by-reference
- Pass by value means that a function can only mutate locations that it is *explicitly given*
    - i.e. as box parameters

The Difference

- Any changes made to the parameter variable are lost in pass-by-value, but kept in pass-by-reference
- Pass by value means that a function can only mutate locations that it is *explicitly given*
  - i.e. as box parameters
- Pass by reference allows you to abstract over patterns of mutation

The Difference

- Any changes made to the parameter variable are lost in pass-by-value, but kept in pass-by-reference
- Pass by value means that a function can only mutate locations that it is *explicitly given*
  - i.e. as box parameters
- Pass by reference allows you to abstract over patterns of mutation
  - e.g. Write a function that says “change these variables in this way” that can be used over and over again

The Difference

- Any changes made to the parameter variable are lost in pass-by-value, but kept in pass-by-reference
- Pass by value means that a function can only mutate locations that it is *explicitly given*
  - i.e. as box parameters
- Pass by reference allows you to abstract over patterns of mutation
  - e.g. Write a function that says “change these variables in this way” that can be used over and over again
  - e.g. swap two variable’s values

Example

Example

```
{let1 {y 2}
  {let1 {f {lam x {seq
                    {set-var! x {* x 3}}
                    x}}}
        {+ {f y} y}}}
```

- In both pass-by-value and pass-by-reference, the call to {f y} produces 6

Example

```
{let1 {y 2}
  {let1 {f {lam x {seq
                    {set-var! x {* x 3}}
                    x}}}
        {+ {f y} y}}}
```

- In both pass-by-value and pass-by-reference, the call to `{f y}` produces 6
- In pass-by-value, `x` has a different location than `y`, that started off with `2` (the value of `y`)

Example

```
{let1 {y 2}
  {let1 {f {lam x {seq
                    {set-var! x {* x 3}}
                    x}}}
        {+ {f y} y}}}
```

- In both pass-by-value and pass-by-reference, the call to `{f y}` produces 6
- In pass-by-value, `x` has a different location than `y`, that started off with `2` (the value of `y`)
  - The final addition adds the value of `{f y}` to the value of `y`, which did not change

Example

```
{let1 {y 2}
  {let1 {f {lam x {seq
                    {set-var! x {* x 3}}
                    x}}}
        {+ {f y} y}}}
```

- In both pass-by-value and pass-by-reference, the call to `{f y}` produces 6
- In pass-by-value, `x` has a different location than `y`, that started off with `2` (the value of `y`)
  - The final addition adds the value of `{f y}` to the value of `y`, which did not change
  - `{+ 6 2}`, result of 8

Example

```
{let1 {y 2}
  {let1 {f {lam x {seq
                    {set-var! x {* x 3}}
                    x}}}
        {+ {f y} y}}}
```

- In both pass-by-value and pass-by-reference, the call to `{f y}` produces 6
- In pass-by-value, `x` has a different location than `y`, that started off with `2` (the value of `y`)
  - The final addition adds the value of `{f y}` to the value of `y`, which did not change
  - `{+ 6 2}`, result of 8
- In pass-by-reference, `x` refers to the same store location as `y`

Example

```
{let1 {y 2}
  {let1 {f {lam x {seq
                    {set-var! x {* x 3}}
                    x}}}
        {+ {f y} y}}}
```

- In both pass-by-value and pass-by-reference, the call to `{f y}` produces 6
- In pass-by-value, `x` has a different location than `y`, that started off with `2` (the value of `y`)
  - The final addition adds the value of `{f y}` to the value of `y`, which did not change
  - `{+ 6 2}`, result of 8
- In pass-by-reference, `x` refers to the same store location as `y`
  - Setting the value of `x` changes `y` because they both refer to the same location

Example

```
{let1 {y 2}
  {let1 {f {lam x {seq
                    {set-var! x {* x 3}}
                    x}}}
        {+ {f y} y}}}
```

- In both pass-by-value and pass-by-reference, the call to `{f y}` produces 6
- In pass-by-value, `x` has a different location than `y`, that started off with `2` (the value of `y`)
  - The final addition adds the value of `{f y}` to the value of `y`, which did not change
  - `{+ 6 2}`, result of 8
- In pass-by-reference, `x` refers to the same store location as `y`
  - Setting the value of `x` changes `y` because they both refer to the same location
  - Result is 12, since both the call and `y` have the value of 6

Which to Choose?

- We will use pass-by-value, since it's simpler

Which to Choose?

- We will use pass-by-value, since it's simpler
  - C++ is pass by value, but sometimes that value is a pointer/reference

Which to Choose?

- We will use pass-by-value, since it's simpler
  - C++ is pass by value, but sometimes that value is a pointer/reference
- However, we can replicate pass-by-reference using Boxes

Which to Choose?

- We will use pass-by-value, since it's simpler
  - C++ is pass by value, but sometimes that value is a pointer/reference
- However, we can replicate pass-by-reference using Boxes
  - This is what e.g. Python does

Which to Choose?

- We will use pass-by-value, since it's simpler
  - C++ is pass by value, but sometimes that value is a pointer/reference
- However, we can replicate pass-by-reference using Boxes
  - This is what e.g. Python does
    - Most objects are implicitly boxed, so it seems like pass by reference

Which to Choose?

- We will use pass-by-value, since it's simpler
  - C++ is pass by value, but sometimes that value is a pointer/reference
- However, we can replicate pass-by-reference using Boxes
  - This is what e.g. Python does
    - Most objects are implicitly boxed, so it seems like pass by reference
    - Actually passing a value, but the value is (something like) a box

Which to Choose?

- We will use pass-by-value, since it's simpler
  - C++ is pass by value, but sometimes that value is a pointer/reference
- However, we can replicate pass-by-reference using Boxes
  - This is what e.g. Python does
    - Most objects are implicitly boxed, so it seems like pass by reference
    - Actually passing a value, but the value is (something like) a box
    - Immutable types (like tuples) are passed by value

Aliasing

- To enable passing by reference, we add an *aliasing expression* to Curly-Mutvar

Aliasing

- To enable passing by reference, we add an *aliasing expression* to Curly-Mutvar
  - Like “address-of” operator & in C++

Aliasing

- To enable passing by reference, we add an *aliasing expression* to Curly-Mutvar
  - Like “address-of” operator & in C++
- `{address-of SYMBOL}` produces a box whose location is the same location as whatever in-scope symbol it is given

Aliasing

- To enable passing by reference, we add an *aliasing expression* to Curly-Mutvar
  - Like “address-of” operator & in C++
- `{address-of SYMBOL}` produces a box whose location is the same location as whatever in-scope symbol it is given
  - e.g. `{address-of x}` would produce `{boxV l}` where `l` is the location in the environment for `x`

Aliasing

- To enable passing by reference, we add an *aliasing expression* to Curly-Mutvar
  - Like “address-of” operator `&` in C++
- `{address-of SYMBOL}` produces a box whose location is the same location as whatever in-scope symbol it is given
  - e.g. `{address-of x}` would produce `{boxV l}` where `l` is the location in the environment for `x`
- Interpreting:

Aliasing

- To enable passing by reference, we add an *aliasing expression* to Curly-Mutvar
  - Like “address-of” operator & in C++
- `{address-of SYMBOL}` produces a box whose location is the same location as whatever in-scope symbol it is given
  - e.g. `{address-of x}` would produce `{boxV l}` where `l` is the location in the environment for `x`
- Interpreting:
  - Like new-box, except we look up the location in the environment instead of getting a new one

Aliasing

- To enable passing by reference, we add an *aliasing expression* to Curly-Mutvar
  - Like “address-of” operator & in C++
- `{address-of SYMBOL}` produces a box whose location is the same location as whatever in-scope symbol it is given
  - e.g. `{address-of x}` would produce `{boxV l}` where `l` is the location in the environment for `x`
- Interpreting:
  - Like new-box, except we look up the location in the environment instead of getting a new one

Aliasing

- To enable passing by reference, we add an *aliasing expression* to Curly-Mutvar
  - Like “address-of” operator & in C++
- `{address-of SYMBOL}` produces a box whose location is the same location as whatever in-scope symbol it is given
  - e.g. `{address-of x}` would produce `{boxV l}` where `l` is the location in the environment for `x`
- Interpreting:
  - Like new-box, except we look up the location in the environment instead of getting a new one

```
[(address-ofE x)
  (v*s (boxV (lookup x env))
    ;; No changes to the store
    sto)]
```

- Now the box and the variable point to the *same location*

Aliasing

- To enable passing by reference, we add an *aliasing expression* to Curly-Mutvar
  - Like “address-of” operator & in C++
- `{address-of SYMBOL}` produces a box whose location is the same location as whatever in-scope symbol it is given
  - e.g. `{address-of x}` would produce `{boxV l}` where `l` is the location in the environment for `x`
- Interpreting:
  - Like new-box, except we look up the location in the environment instead of getting a new one

```
[(address-ofE x)
  (v*s (boxV (lookup x env))
    ;; No changes to the store
    sto)]
```

- Now the box and the variable point to the *same location*
- Changes to one are seen in the other

Example

Example

```
{let1 {doublebox! {lam x {set-box! x {* 2 {unbox x}}}}}
  {let1 {y 3}
            {seq {doublebox! {address-of y}}
                 y}}}
```

- Function takes in a box, gets its value, doubles it, and writes it to the same location

Example

```
{let1 {doublebox! {lam x {set-box! x {* 2 {unbox x}}}}}
  {let1 {y 3}
            {seq {doublebox! {address-of y}}
                 y}}}
```

- Function takes in a box, gets its value, doubles it, and writes it to the same location
- The `address-of` makes a new box whose location is the same as `y`

Example

```
{let1 {doublebox! {lam x {set-box! x {* 2 {unbox x}}}}}
  {let1 {y 3}
            {seq {doublebox! {address-of y}}
                 y}}}
```

- Function takes in a box, gets its value, doubles it, and writes it to the same location
- The `address-of` makes a new box whose location is the same as `y`
- When `doublebox!` runs it alters the value at the location of its box, which was the location of `y`

Example

```
{let1 {doublebox! {lam x {set-box! x {* 2 {unbox x}}}}}
  {let1 {y 3}
            {seq {doublebox! {address-of y}}
                 y}}}
```

- Function takes in a box, gets its value, doubles it, and writes it to the same location
- The `address-of` makes a new box whose location is the same as `y`
- When `doublebox!` runs it alters the value at the location of its box, which was the location of `y`
- The final result is 6, since the value of `y` was changed

Static Scope and Mutable Variables

- Mutable variables are still statically scoped

Static Scope and Mutable Variables

- Mutable variables are still statically scoped
  - Function is executed in the environment from when it was defined

Static Scope and Mutable Variables

- Mutable variables are still statically scoped
  - Function is executed in the environment from when it was defined
  - BUT that environment maps symbols to locations

Static Scope and Mutable Variables

- Mutable variables are still statically scoped
  - Function is executed in the environment from when it was defined
  - BUT that environment maps symbols to locations
    - The location's value might change after the function is defined

Static Scope and Mutable Variables

- Mutable variables are still statically scoped
  - Function is executed in the environment from when it was defined
  - BUT that environment maps symbols to locations
    - The location's value might change after the function is defined
    - But the symbols still map to the same locations

Static Scope and Mutable Variables

- Mutable variables are still statically scoped
  - Function is executed in the environment from when it was defined
  - BUT that environment maps symbols to locations
    - The location's value might change after the function is defined
    - But the symbols still map to the same locations

Static Scope and Mutable Variables

- Mutable variables are still statically scoped
  - Function is executed in the environment from when it was defined
  - BUT that environment maps symbols to locations
    - The location's value might change after the function is defined
    - But the symbols still map to the same locations

```
{let1 {x 10}
  {let1 {f {lam y {* x y}}}}
    {seq {set-var! x 20}
         {let1 {x 100}
               {f 5}}
  }}
```

- In the function body, x has location from point of definition

Static Scope and Mutable Variables

- Mutable variables are still statically scoped
  - Function is executed in the environment from when it was defined
  - BUT that environment maps symbols to locations
    - The location's value might change after the function is defined
    - But the symbols still map to the same locations

```
{let1 {x 10}
  {let1 {f {lam y {* x y}}}}
    {seq {set-var! x 20}
         {let1 {x 100}
               {f 5}}
  }}
```

- In the function body, `x` has location from point of definition
  - This location starts with `10`

Static Scope and Mutable Variables

- Mutable variables are still statically scoped
  - Function is executed in the environment from when it was defined
  - BUT that environment maps symbols to locations
    - The location's value might change after the function is defined
    - But the symbols still map to the same locations

```
{let1 {x 10}
  {let1 {f {lam y {* x y}}}}
    {seq {set-var! x 20}
         {let1 {x 100}
               {f 5}}
  }}
```

- In the function body, `x` has location from point of definition
  - This location starts with `10`
  - `set-var` updates its value to `20`

Static Scope and Mutable Variables

- Mutable variables are still statically scoped
  - Function is executed in the environment from when it was defined
  - BUT that environment maps symbols to locations
    - The location's value might change after the function is defined
    - But the symbols still map to the same locations

```
{let1 {x 10}
  {let1 {f {lam y {* x y}}}}
    {seq {set-var! x 20}
         {let1 {x 100}
               {f 5}}
  }}
```

- In the function body, `x` has location from point of definition
  - This location starts with `10`
  - `set-var` updates its value to `20`
- A new variable `x` is defined for the scope that `f` is called

Static Scope and Mutable Variables

- Mutable variables are still statically scoped
  - Function is executed in the environment from when it was defined
  - BUT that environment maps symbols to locations
    - The location's value might change after the function is defined
    - But the symbols still map to the same locations

```
{let1 {x 10}
  {let1 {f {lam y {* x y}}}}
    {seq {set-var! x 20}
         {let1 {x 100}
               {f 5}}
  }}
```

- In the function body, `x` has location from point of definition
  - This location starts with `10`
  - `set-var` updates its value to `20`
- A new variable `x` is defined for the scope that `f` is called
  - Its location contains `100`

Static Scope and Mutable Variables

- Mutable variables are still statically scoped
  - Function is executed in the environment from when it was defined
  - BUT that environment maps symbols to locations
    - The location's value might change after the function is defined
    - But the symbols still map to the same locations

```
{let1 {x 10}
  {let1 {f {lam y {* x y}}}}
    {seq {set-var! x 20}
         {let1 {x 100}
               {f 5}}
  }}
```

- In the function body, `x` has location from point of definition
  - This location starts with `10`
  - `set-var` updates its value to `20`
- A new variable `x` is defined for the scope that `f` is called
  - Its location contains `100`
  - NOT used in the function call, because not in scope for function definition

Evaluation styles

- 3 different properties of an interpreter

Evaluation styles

- 3 different properties of an interpreter
  - Strict vs lazy

Evaluation styles

- 3 different properties of an interpreter
  - Strict vs lazy
    - Are function arguments evaluated before substituting or putting in environment

Evaluation styles

- 3 different properties of an interpreter
  - Strict vs lazy
    - Are function arguments evaluated before substituting or putting in environment
  - Static scope vs. dynamic scope

Evaluation styles

- 3 different properties of an interpreter
  - Strict vs lazy
    - Are function arguments evaluated before substituting or putting in environment
  - Static scope vs. dynamic scope
    - In a function call, are free variables looked-up in the environment the function definition environment, or the function call environment?

Evaluation styles

- 3 different properties of an interpreter
  - Strict vs lazy
    - Are function arguments evaluated before substituting or putting in environment
  - Static scope vs. dynamic scope
    - In a function call, are free variables looked-up in the environment the function definition environment, or the function call environment?
  - Call-by-value vs. Call-by-reference

Evaluation styles

- 3 different properties of an interpreter
  - Strict vs lazy
    - Are function arguments evaluated before substituting or putting in environment
  - Static scope vs. dynamic scope
    - In a function call, are free variables looked-up in the environment the function definition environment, or the function call environment?
  - Call-by-value vs. Call-by-reference
    - Do function calls on variables create new memory locations, or new symbols pointing to the same location as the argument variables?

Evaluation styles

- 3 different properties of an interpreter
  - Strict vs lazy
    - Are function arguments evaluated before substituting or putting in environment
  - Static scope vs. dynamic scope
    - In a function call, are free variables looked-up in the environment the function definition environment, or the function call environment?
  - Call-by-value vs. Call-by-reference
    - Do function calls on variables create new memory locations, or new symbols pointing to the same location as the argument variables?
- These properties are **completely separate from each other**

Evaluation styles

- 3 different properties of an interpreter
  - Strict vs lazy
    - Are function arguments evaluated before substituting or putting in environment
  - Static scope vs. dynamic scope
    - In a function call, are free variables looked-up in the environment the function definition environment, or the function call environment?
  - Call-by-value vs. Call-by-reference
    - Do function calls on variables create new memory locations, or new symbols pointing to the same location as the argument variables?
- These properties are **completely separate from each other**
  - You can mix them in any combination

Evaluation styles

- 3 different properties of an interpreter
  - Strict vs lazy
    - Are function arguments evaluated before substituting or putting in environment
  - Static scope vs. dynamic scope
    - In a function call, are free variables looked-up in the environment the function definition environment, or the function call environment?
  - Call-by-value vs. Call-by-reference
    - Do function calls on variables create new memory locations, or new symbols pointing to the same location as the argument variables?
- These properties are **completely separate from each other**
  - You can mix them in any combination
  - But lazy + mutable state is generally a bad idea